

Cuadro comparativo – Diseño UML propio vs diseño sugerido por IA

Prompt utilizado para solicitar revisión a la IA

Actúa como Arquitecto de Software Principal y Consultor Senior de Java. Analiza el diagrama UML de un sistema hotelero y propone mejoras aplicando principios de Programación Orientada a Objetos, herencia, abstracción, encapsulamiento, composición, agregación y persistencia de datos.

Resumen de la recomendación realizada por IA

La IA propuso separar el sistema en capas, implementar clases abstractas, utilizar herencia, incorporar DAOs, enums y mejorar la modularidad.

Tabla comparativa

Elemento analizado	Diseño con IA	Diseño sin IA	Decisión final del grupo	Justificación
Estructura general del sistema	Arquitectura dividida en modelo, DAO y GUI	Diseño centrado únicamente en clases principales del negocio	Se adoptó una estructura parcialmente por capas	Facilita organización y mantenimiento del sistema
Clase Cliente	Clase abstracta Cliente con herencia	Cliente manejado de forma más simple	Se mantuvo Cliente abstracto	Permite reutilizar atributos y métodos comunes
Tipos de cliente	PersonaNatural y Empresa con subtipos	Menor nivel de especialización	Se conservaron los subtipos principales	Mejora flexibilidad y manejo de descuentos
Clase Habitación	Clase abstracta con herencia	Habitaciones modeladas de forma básica	Se adoptó herencia en habitaciones	Permite extender tipos de habitación fácilmente
Tipos de habitación	Sencilla, Doble y Matrimonial	Menor diferenciación de atributos	Se conservaron tipos específicos	Representa mejor la realidad del hotel
Gestión de reservas	Clase Reserva centralizada y relacionada	Reserva menos integrada	Se mantuvo modelo centralizado	Facilita control del flujo del sistema
Servicios adicionales	Inclusión de servicios	Manejo limitado de	Se agregaron servicios	Permite calcular costos

	consumidos	servicios	consumidos	adicionales
Facturación	Clase Factura integrada	Facturación menos detallada	Se añadió facturación formal	Mejora control financiero
Persistencia de datos	Uso de clases DAO	No existía capa DAO	Se adoptó parcialmente DAO	Favorece separación entre lógica y almacenamiento
Estados del sistema	Uso de enums	Estados manuales	Se incorporaron enums	Reduce errores y mejora claridad
Interfaces y modularidad	Uso de interfaces/capas DAO y separación modular	Diseño más directo y menos desacoplado	Se adoptó parcialmente la modularidad propuesta	Mejora mantenibilidad y escalabilidad
Patrón de diseño Strategy (Fidelidad)	Interfaz IFidelidad con implementaciones ClienteHabitual y ClienteEsporadico aplicando patrón Strategy para calcular descuentos	No se aplicó patrón Strategy; los tipos de cliente se modelaron sin una interfaz común de fidelidad	Se evaluó adoptar el patrón Strategy para el cálculo de descuentos según tipo de cliente	Permite cambiar la política de descuentos sin modificar la clase Cliente; mejora extensibilidad
Enumeraciones específicas (detalle)	Enums concretos: EstadoHabitacion (DISPONIBLE, OCUPADO, LIMPIEZA, RESERVADA), EstadoReserva (PENDIENTE, CONFIRMADA, CANCELADA, FINALIZADA) y MetodoPersistencia (ARCHIVO_BINARIO, SERIALIZACION)	Los estados se representaban como String sin validación de valores posibles	Se incorporaron los enums EstadoHabitacion y EstadoReserva; MetodoPersistencia según método elegido	Evita valores inválidos en tiempo de ejecución y mejora la legibilidad del código
Capa de interfaz gráfica (GUI)	Clases explícitas de GUI: VentanaClientes, VentanaHabitacione	No se modeló la capa de presentación	Se adoptó la separación de ventanas por funcionalidad	Representa la capa de presentación de forma

	s, VentanaReservas, VentanaServicios, VentanaFacturacion y SistemaHotelGUI como punto de entrada	en el diagrama	tal como propone la IA	explícita y facilita la asignación de responsabilidades por módulo
Tipos de datos para valores monetarios y fechas	Uso de BigDecimal para precios, descuentos y subtotales; LocalDate para manejo de fechas	Uso de double para valores monetarios y String para fechas	Se adoptó BigDecimal para cálculos financieros y LocalDate para fechas de reserva y factura	BigDecimal evita errores de precisión en operaciones financieras; LocalDate es más seguro y expresivo que String
Método de persistencia específico	Enum MetodoPersistencia con opciones ARCHIVO_BINARIO y SERIALIZACION; DAOs con métodos guardar, buscar, listar y eliminar por entidad	Persistencia centralizada en la clase Hotel con métodos guardarDatos y cargarDatos sin distinción de mecanismo	Se adoptó serialización como método de persistencia con DAOs por entidad (ClienteDAO, ReservaDAO, HabitacionDAO, etc.)	Define claramente el mecanismo de almacenamiento y distribuye la responsabilidad de persistencia por entidad
Responsabilidades de la clase Hotel	Hotel actúa como fachada; delega operaciones a DAOs y ventanas especializadas; menos métodos directos	Hotel concentra la mayoría de la lógica del sistema: registro, búsqueda, listado y generación de facturas	Se distribuyó parte de la lógica hacia DAOs manteniendo en Hotel solo la coordinación general	Reduce el acoplamiento y aplica el principio de responsabilidad única (SRP)

Explicación breve de las decisiones de diseño

El diseño final consolida una arquitectura robusta que separa la interfaz, la lógica y los datos. Integra clases abstractas para **Cliente** y **Habitación**, permitiendo una especialización clara de sus subtipos. Se incluyen capas **DAO** para la persistencia y el uso de **Enums** para el control estricto de estados.

El diseño se basa en los siguientes pilares de programación orientada a objetos:

- **Abstracción y Herencia:** Se implementaron clases abstractas para Cliente y Habitación. Esto permite reutilizar atributos comunes y extender el sistema con nuevos tipos de clientes o habitaciones sin alterar la base del código.
- **Encapsulamiento y Modularidad:** El uso de la arquitectura **DAO** separa la lógica del negocio del almacenamiento, mejorando la mantenibilidad y escalabilidad.
- **Relaciones y Composición:** La clase Reserva centraliza la relación entre clientes, habitaciones y servicios adicionales. Se aplicó el principio de **Responsabilidad Única (SRP)** delegando funciones específicas a componentes especializados.
- **Polimorfismo:** El patrón **Strategy** (Fidelidad) permite aplicar diferentes lógicas de descuento de forma dinámica según el tipo de cliente.
- **Robustez de Datos:** Se reemplazó el uso de double por BigDecimal para garantizar precisión financiera y LocalDate para un manejo seguro de fechas.